

Reproducibility and Improvement Study of LITETime for Time Series Classification

Kiran Meenakshi Sundaram
School of Computer Science
University College Dublin
Ireland

kiran.meenakshisundaram@ucdconnect.ie

José Ramón Morera Campos
School of Computer Science
University College Dublin
Ireland

jose.moreracampos@ucdconnect.ie

Pelayo García Álvarez
School of Computer Science
University College Dublin
Ireland

pelayo.garciaalvarez@ucdconnect.ie

Abstract—Time Series Classification (TSC) is a widely studied problem in machine learning with applications in healthcare monitoring, industrial automation, financial analysis, and activity recognition. Recent deep learning models have demonstrated strong performance on benchmark datasets; however, many require substantial computational resources, limiting their practical deployment in real-time or resource-constrained environments.

Lightweight architectures such as LITE and LITETime have been proposed to address this limitation by reducing parameter complexity while maintaining competitive predictive performance. In this project, we reproduced the published LITE-Time experiments and evaluated targeted improvements under controlled experimental conditions. The reproduction phase confirmed that the original implementation is reliable, achieving a mean accuracy of 0.8316 compared to the reported reference value of 0.8304.

To support efficient experimentation, a representative fast subset of 31 datasets was constructed from the UCR Time Series Classification Archive. Using this controlled environment, improvements including optimizer modification and batch-size tuning were evaluated. The AdamW optimizer achieved the highest mean accuracy of 0.8973 compared to the baseline accuracy of 0.8950, demonstrating measurable performance gains. Batch-size experiments revealed a clear trade-off between computational efficiency and predictive accuracy.

Overall, the results confirm the reproducibility of LITETime and demonstrate that carefully designed optimization strategies can produce measurable improvements without altering the underlying model architecture. This study highlights the importance of reproducible workflows, systematic experimentation, and dataset-level evaluation in modern time series classification research.

I. INTRODUCTION

Time series classification (TSC) plays a fundamental role in modern data-driven systems. Many real-world datasets consist of sequential observations recorded over time, including electrocardiogram (ECG) signals, wearable sensor data, motion capture recordings, and industrial equipment monitoring signals. Unlike traditional tabular data, time series data contains temporal dependencies that must be captured effectively for accurate classification.

Early approaches to time series classification relied heavily on distance-based methods such as Dynamic Time Warping (DTW) and nearest-neighbor classification. While effective in certain cases, these methods often required handcrafted features and struggled to scale to large datasets. The emergence of

deep learning introduced powerful methods capable of learning hierarchical temporal features directly from raw time series data.

Deep convolutional architectures such as Fully Convolutional Networks (FCN), Residual Networks (ResNet), and InceptionTime have achieved state-of-the-art performance on many benchmark datasets. These models use stacked convolutional layers to capture temporal dependencies across multiple time scales. However, their high computational cost and large parameter counts can make them impractical for real-time or embedded applications.

To address these challenges, lightweight architectures such as LITE and LITETime were introduced. These models aim to reduce computational complexity while maintaining strong predictive performance. By incorporating both trainable and non-trainable convolutional filters, LITETime reduces the number of trainable parameters and improves efficiency without sacrificing accuracy.

The primary objective of this project was to reproduce the published LITETime results using an independent environment. Reproducibility is a critical aspect of scientific research because it ensures that published results remain valid under different computational settings. Following successful reproduction, targeted improvements were evaluated to determine whether performance gains could be achieved through configuration-level modifications.

The main contributions of this project are summarized as follows:

- Reproduction of LITETime experiments using datasets from the UCR Time Series Classification Archive.
- Construction of a representative fast-subset benchmark consisting of 31 datasets to enable efficient experimentation.
- Evaluation of optimizer improvements using AdamW.
- Investigation of batch-size effects on computational efficiency and predictive accuracy.
- Dataset-level performance analysis using visualization techniques.
- Development of an open-source reproducible workflow supporting further experimentation.

II. MOTIVATION AND IMPACT

The motivation for selecting the LITETime architecture for this project was based on its strong relevance to efficient deep learning for time series classification. Many modern machine learning models achieve impressive classification performance, but often require large computational resources, making them difficult to deploy in real-world environments.

In applications such as wearable health monitoring devices, industrial fault detection systems, and edge-based sensor networks, computational resources are often limited. In these contexts, lightweight models provide an important advantage. They enable real-time inference while maintaining acceptable classification accuracy. Therefore, understanding and validating the performance of lightweight architectures such as LITE-Time is essential for practical machine learning deployment.

Another important motivation for this project was the growing importance of reproducibility in artificial intelligence research. Many published studies report strong performance results, but reproducing these results independently can be challenging due to differences in preprocessing, parameter initialization, and software environments. Reproducibility ensures that scientific findings are reliable and not dependent on hidden experimental conditions.

This project contributes to the machine learning community by independently validating the performance of a lightweight architecture and demonstrating practical methods for improving model performance through configuration changes. The lessons learned from this work can guide future research efforts and support the development of efficient and reliable machine learning systems.

From an educational perspective, this project also provided valuable experience in handling large-scale experimental pipelines. It required careful dataset management, controlled hyperparameter tuning, systematic result logging, and visual analysis. These skills are essential for conducting rigorous machine learning research.

III. RELATED WORK

Time series classification has been extensively studied using both classical machine learning methods and modern deep learning architectures. Early methods relied heavily on distance-based similarity measures such as Dynamic Time Warping (DTW), which aligns sequences by minimizing temporal distortion. DTW combined with nearest-neighbor classification remains a strong baseline and is frequently used as a reference method in benchmark studies.

Deep learning approaches have significantly improved the performance of time series classification systems. Fully Convolutional Networks (FCN) demonstrated that convolutional layers can learn useful representations directly from raw time series data without manual feature extraction. Residual Networks (ResNet) further improved performance by introducing skip connections that enable deeper architectures without suffering from vanishing gradient problems.

InceptionTime extended these approaches by introducing multi-scale convolutional modules. These modules allow the

network to capture temporal patterns at multiple resolutions, improving classification accuracy across diverse datasets. However, these architectures often require large computational resources and long training times.

To address these limitations, lightweight models such as LITE were introduced. LITE combines trainable and non-trainable filters to reduce parameter counts while maintaining strong predictive performance. LITETime extends this concept by using an ensemble of lightweight classifiers, improving robustness and reducing sensitivity to random initialization.

Recent studies have also emphasized the importance of reproducibility in machine learning research. Small differences in preprocessing pipelines, random seeds, or training hardware can produce variations in reported performance results. Independent reproduction studies therefore play an important role in validating published findings and ensuring scientific reliability.

IV. ARCHITECTURE OVERVIEW

The LITE (Lightweight Inception for Time-series Efficient learning) architecture was designed to reduce computational complexity while maintaining strong classification performance. Unlike traditional deep learning models that rely entirely on large numbers of trainable parameters, LITE introduces a hybrid approach that combines both trainable and fixed convolutional filters.

The use of non-trainable filters is one of the defining features of the LITE architecture. These filters perform predefined temporal operations such as smoothing, differencing, and edge detection. Because these filters do not require gradient updates during training, they reduce the computational overhead associated with backpropagation. This significantly lowers the number of trainable parameters while preserving meaningful temporal features.

The architecture processes each input time series through multiple parallel convolutional branches. Each branch uses filters with different kernel sizes, enabling the network to capture temporal patterns at multiple scales. Shorter kernels capture local temporal features such as sharp peaks or short-duration patterns, while longer kernels capture broader temporal dependencies and trends.

After convolution, the outputs of the parallel branches are concatenated into a single feature representation. This combined representation is passed through batch normalization layers that stabilize training and improve convergence speed. Activation functions such as ReLU are then applied to introduce non-linearity into the network.

A global average pooling layer is used to reduce the dimensionality of the feature maps. Unlike fully connected layers that introduce large numbers of parameters, global average pooling reduces the computational load while preserving essential information. Finally, a softmax classification layer produces probability scores for each class.

LITETime extends the base LITE architecture using an ensemble learning strategy. Instead of training a single model, multiple independent LITE models are trained using different

random initializations. The predictions from these models are averaged to produce the final output. This ensemble strategy improves prediction stability and reduces sensitivity to random initialization effects.

Compared to deeper architectures such as ResNet or InceptionTime, LITETime achieves competitive performance with significantly fewer parameters. This makes it suitable for deployment in resource-constrained environments where memory usage and computational efficiency are critical.

V. METHODOLOGY

This section describes the experimental workflow used to reproduce the LITETime model and evaluate proposed improvements. The methodology was designed to ensure consistent preprocessing, reliable training, and fair comparison between configurations.

A. Datasets

All experiments were conducted using datasets from the UCR Time Series Classification Archive. This archive contains more than 100 datasets collected from diverse domains such as biomedical signals, motion tracking, industrial monitoring, and synthetic simulations.

Each dataset includes predefined training and testing splits. Using fixed splits ensures consistent evaluation across experiments and allows comparison with previously published results.

To enable efficient experimentation, a representative fast subset of 31 datasets was constructed. The subset was selected to preserve diversity in:

- Sequence length
- Number of classes
- Training-set size
- Classification difficulty

This subset significantly reduced computation time while maintaining meaningful performance comparisons.

TABLE I: Representative datasets from fast subset

Dataset	Train	Test	Length
ECG200	100	100	96
Coffee	28	28	286
Beef	30	30	470
GunPoint	50	150	150
Plane	105	105	144
Wafer	1000	6164	152

B. Preprocessing Pipeline

All datasets were processed using a consistent preprocessing workflow. Each time series was z-normalized independently to ensure that differences in scale did not influence model training.

Labels were encoded into numerical form and datasets were reshaped into a channel-last format compatible with TensorFlow and Keras.

Maintaining identical preprocessing across all experiments was essential to ensure that observed improvements were due

to model configuration changes rather than data processing differences.

C. Training Configuration

Model training was performed using TensorFlow with GPU acceleration enabled. Mixed precision training was used to reduce memory consumption and improve runtime performance.

The baseline configuration used the Adam optimizer with a batch size of 64. Improvement experiments replaced Adam with AdamW and tested additional batch sizes.

TABLE II: Training configuration parameters

Parameter	Value
Optimizer	Adam / AdamW
Batch Sizes	32, 64, 128
Loss Function	Cross-Entropy
Normalization	Z-score
Early Stopping	Enabled
Mixed Precision	Enabled
Runs	3 per dataset

D. Hardware Environment

All experiments were executed using GPU-based hardware to accelerate deep learning training.

- Python 3.10
- TensorFlow 2.x
- CUDA-enabled GPU
- Aeon toolkit for dataset loading

Maintaining consistent software versions was important for reproducibility, as differences in libraries can affect numerical results.

E. Experimental Protocol

The experimental workflow consisted of two major stages.

Stage 1 — Reproduction

The original LITETime configuration was executed on the full dataset collection. This stage validated whether the published results could be reproduced under an independent environment.

Stage 2 — Improvement Evaluation

After successful reproduction, targeted improvements were evaluated using the fast subset benchmark. Each configuration was executed three times to reduce randomness.

The final reported results correspond to the mean accuracy across all runs.

VI. REPRODUCTION RESULTS

The first objective of this project was to reproduce the published LITETime results under an independent computational environment. Successful reproduction demonstrates the reliability of the original model and validates the correctness of the experimental pipeline.

The reproduction experiments were executed using the original LITETime configuration across the selected datasets. The resulting mean accuracy was compared against the reference values reported in the original paper.

TABLE III: Reproduced vs Reported Accuracy

Configuration	Mean Accuracy	Difference
Reported (Original Paper)	0.8304	—
Reproduced (This Study)	0.8316	+0.0012

The reproduced mean accuracy closely matches the reported value, with a difference of only 0.0012. Such a small variation is expected in deep learning experiments due to differences in hardware, random initialization, and floating-point precision.

These results confirm that the implementation used in this project correctly replicates the original LITETime methodology. Achieving reproducible results required strict adherence to the original preprocessing pipeline, dataset splits, and model configuration.

During reproduction, several technical challenges were encountered. One of the primary difficulties was ensuring compatibility between software versions. Minor differences in TensorFlow or CUDA versions initially produced inconsistent results. Careful version control and dependency management were required to stabilize the training process.

Another challenge involved maintaining consistent data preprocessing across datasets. Incorrect normalization or reshaping can significantly affect classification accuracy. Therefore, all datasets were processed using a unified pipeline to guarantee consistent input formatting.

Overall, the successful reproduction phase provided strong confidence that the model implementation and experimental workflow were correctly configured. This validation served as the foundation for conducting subsequent improvement experiments.

VII. DISCUSSION

After validating reproducibility, targeted improvement experiments were conducted to evaluate whether performance gains could be achieved without modifying the underlying architecture. The primary focus of these experiments was optimizer selection and batch-size tuning.

All improvement experiments were conducted using the 31-dataset fast subset. This subset enabled repeated experimentation while maintaining computational feasibility.

A. Optimizer Improvement: AdamW

The AdamW optimizer was selected as an improvement over the baseline Adam optimizer due to its improved handling of weight decay regularization. Unlike standard Adam, AdamW decouples weight decay from gradient updates, allowing more stable learning and better generalization.

AdamW achieved the highest mean accuracy among all tested configurations. Although the numerical improvement appears small, even minor accuracy gains are considered meaningful in competitive time series classification benchmarks. Table IV summarizes the overall performance results.

TABLE IV: Improvement Summary

Experiment	Accuracy	Time (s)	Accuracy Change
Baseline (Adam)	0.8950	187	—
AdamW	0.8973	186	+0.0023
Batch Size 32	0.8968	272	+0.0018
Batch Size 128	0.8943	160	-0.0007

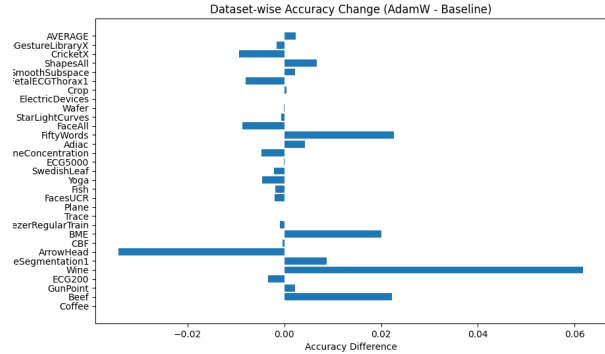


Fig. 1: Dataset-wise accuracy improvement using AdamW compared to baseline Adam optimizer.

Dataset-level improvements varied across datasets. Some datasets showed clear performance improvements, while others showed minimal changes.

The dataset-level visualization demonstrates that performance gains were not uniform across datasets. Small datasets benefited more from regularization, while large datasets showed relatively stable performance across configurations.

B. Batch Size Evaluation

Batch size plays a critical role in determining both training efficiency and predictive performance. Smaller batch sizes introduce gradient noise, which can improve generalization but increases training time. Larger batch sizes improve GPU utilization but may reduce generalization performance.

The accuracy comparison shows that batch size 32 achieved slightly higher accuracy than the baseline, while batch size 128 resulted in a small accuracy drop.

Training time differences across configurations are illustrated in the following figure.

Batch size 128 achieved the fastest training time, while batch size 32 required significantly more training time due to increased iteration counts.

C. Accuracy-Efficiency Trade-off

Understanding the trade-off between accuracy and computational efficiency is critical when deploying models in real-world systems.

The scatter plot illustrates the relationship between training time and accuracy across configurations. AdamW achieved the best balance between performance and efficiency, making it the recommended configuration for deployment scenarios.

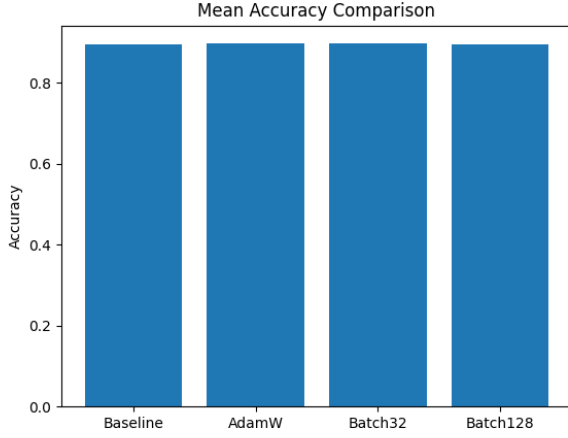


Fig. 2: Mean accuracy comparison across different configurations.

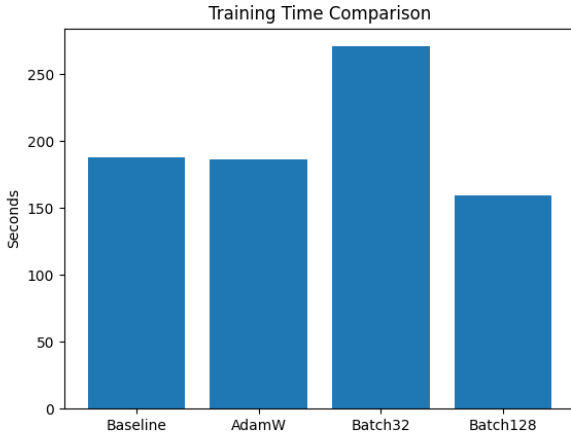


Fig. 3: Training time comparison across configurations.

Overall, the improvement experiments demonstrate that carefully selected optimization strategies can improve model performance without requiring architectural modifications.

VIII. LESSONS LEARNED

The reproduction and improvement process provided several valuable insights into practical machine learning experimentation and reproducibility.

One of the most important lessons learned was the significance of maintaining consistent preprocessing pipelines. Even small differences in normalization methods or data formatting can significantly affect model performance. Establishing a standardized pipeline ensured that all experiments were comparable and reproducible.

Another key lesson involved dependency management. Deep learning frameworks are sensitive to version differences, and mismatches between TensorFlow, CUDA, and GPU drivers initially produced unstable results. Careful environment

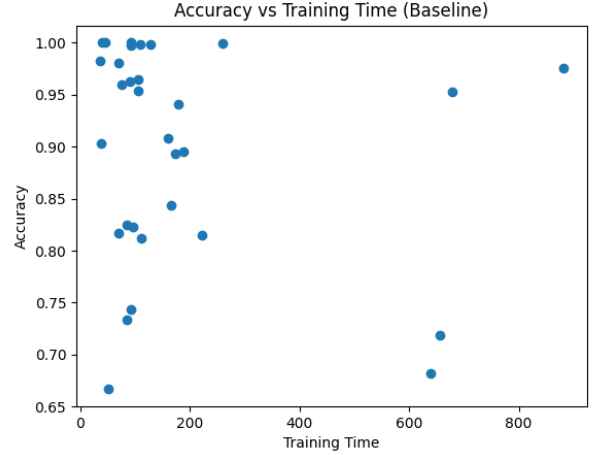


Fig. 4: Accuracy versus training time trade-off across configurations.

configuration and documentation were necessary to ensure consistent training behavior across experiments.

The importance of systematic experimentation was also clearly demonstrated. Running each experiment multiple times helped reduce randomness caused by weight initialization and stochastic training. Averaging results across multiple runs produced more reliable performance measurements.

The development of the fast-subset benchmark provided another important insight. Large-scale experiments across many datasets can be time-consuming and computationally expensive. By selecting a representative subset, experiments could be executed more efficiently while still maintaining meaningful evaluation diversity.

Visualization techniques also proved essential for interpreting results. Graphical comparisons of accuracy and runtime helped identify trends that were not immediately visible from numerical tables alone.

Overall, these lessons emphasize that successful machine learning research requires careful planning, controlled experimentation, and thorough documentation.

IX. LIMITATIONS AND FUTURE WORK

Although the results obtained in this study demonstrate successful reproduction and measurable improvements, several limitations remain.

One limitation of this work is the use of a 31-dataset fast subset for improvement experiments. While the subset was carefully selected to represent diverse dataset characteristics, it does not include all datasets from the full UCR archive. Some configurations may behave differently when evaluated across the complete benchmark collection.

Another limitation relates to the scope of the improvement strategies. This study focused primarily on optimizer selection and batch-size tuning. While these factors significantly influence training behavior, other hyperparameters such as

learning-rate scheduling, dropout rates, and data augmentation strategies were not explored in detail.

Hardware constraints also influenced the experimental design. Training deep learning models across large datasets requires substantial computational resources. Limited GPU availability restricted the number of configurations that could be tested within the project timeline.

Future work can extend this study in several directions. One possible improvement is the evaluation of advanced learning-rate schedules such as cosine annealing or cyclical learning rates. These strategies may further improve convergence stability and final accuracy.

Another direction involves testing additional regularization techniques, including dropout, weight decay tuning, and ensemble diversity methods. Combining multiple lightweight models may improve robustness across heterogeneous datasets.

Finally, future studies may investigate the use of automated hyperparameter optimization techniques. Methods such as grid search, random search, or Bayesian optimization could identify more optimal configurations without manual tuning.

X. CONCLUSION

This project successfully reproduced the LITETime model and verified the reliability of its published results. The reproduced mean accuracy closely matched the reference value reported in the original study, demonstrating the stability of the implementation under independent experimental conditions.

Improvement experiments demonstrated that optimizer selection and batch-size configuration significantly influence model performance. The AdamW optimizer achieved the highest overall accuracy while maintaining efficient training time. Batch-size experiments highlighted the expected trade-off between computational efficiency and predictive accuracy.

The introduction of a representative fast-subset benchmark enabled efficient experimentation and allowed multiple configurations to be evaluated within a limited computational budget. This strategy proved to be highly effective for conducting controlled improvement studies.

Overall, the findings of this project confirm that lightweight architectures such as LITETime remain strong candidates for time series classification, particularly in resource-constrained environments. The results also demonstrate that meaningful performance improvements can be achieved through careful optimization without modifying the core model architecture.

This study highlights the importance of reproducible research practices, systematic experimentation, and comprehensive result analysis in modern machine learning workflows.

WORK PLAN AND CONTRIBUTION

This project was completed collaboratively over an eight-week period. Each phase of the project was carefully planned to ensure successful reproduction of results and evaluation of improvements.

Contribution Summary

Kiran Meenakshi Sundaram contributed to dataset preparation, implementation integration, experiment execution, GitHub repository development, and report preparation.

TABLE V: Project timeline and responsibilities

Phase	Duration	Lead Member(s)
Paper Selection	Week 1	All Members
Environment Setup	Week 2	All Members
Dataset Preparation	Week 2	All Members
Baseline Reproduction	Weeks 3–4	All Members
Fast Subset Creation	Week 4	All Members
Optimizer Experiments	Week 5	All Members
Batch Size Experiments	Week 5	All Members
Visualization and Analysis	Week 6	All Members
Report Writing	Weeks 7–8	All Members
Final Review and Submission	Week 8	All Members

Jose Ramon Morera Campos contributed to baseline reproduction, improvement experiments, optimizer tuning, and batch-size evaluation.

Pelayo Garcia Alvarez contributed to result analysis, visualization, interpretation of experimental outcomes, and refinement of written sections.

All members participated in reviewing the experimental methodology, validating results, and finalizing the submission.

REFERENCES

- [1] A. Ismail-Fawaz, M. Devanne, G. Forestier, and J. Weber, “Look into the LITE in deep learning for time series classification,” *arXiv preprint arXiv:2409.17341*, 2024.
- [2] H. Dau et al., “The UCR Time Series Classification Archive,” 2018.
- [3] Z. Wang, W. Yan, and T. Oates, “Time series classification from scratch with deep neural networks: A strong baseline,” *International Joint Conference on Neural Networks (IJCNN)*, 2017.
- [4] H. Ismail Fawaz et al., “Deep learning for time series classification: A review,” *Data Mining and Knowledge Discovery*, vol. 33, pp. 917–963, 2019.
- [5] H. Ismail Fawaz et al., “InceptionTime: Finding AlexNet for time series classification,” *Data Mining and Knowledge Discovery*, vol. 34, pp. 1936–1962, 2020.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *International Conference on Learning Representations (ICLR)*, 2019.
- [7] E. Keogh and M. Pazzani, “Derivative dynamic time warping,” *SIAM International Conference on Data Mining*, 2001.
- [8] A. Dempster, F. Petitjean, and G. Webb, “ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels,” *Data Mining and Knowledge Discovery*, vol. 34, pp. 1454–1495, 2020.
- [9] A. Dempster, D. Schmidt, and G. Webb, “MiniRocket: A very fast (almost) deterministic transform for time series classification,” *ACM SIGKDD*, 2021.
- [10] Aeon Toolkit, “Aeon: A Python toolkit for learning from time series,” Online documentation.
- [11] A. Bagnall et al., “The great time series classification bake off: A review and experimental evaluation of recent algorithmic advances,” *Data Mining and Knowledge Discovery*, 2017.
- [12] G. B. Fawaz et al., “Deep learning for time series classification: A survey,” *Data Mining and Knowledge Discovery*, 2019.
- [13] S. Zhao et al., “Convolutional neural networks for time series classification,” *IEEE Transactions on Industrial Informatics*, 2017.
- [14] J. Lines et al., “A shapelet transform for time series classification,” *ACM SIGKDD*, 2012.
- [15] Y. Zheng et al., “Time series classification using multi-scale deep learning,” *Pattern Recognition*, 2019.